```
    define_method "#{attribute}=" do |value|
      # ...
    end

    define_method attribute do
      # ...
    end
  end
end
```

The previous code defines two *Mimic Methods (218)* that are supposed to read and write an instance variable. How can the code do this without evaluating a String of Code? If you browse through Ruby's documentation, you'll find a few methods that manipulate instance variables, including Object#instance_variable_get and Object#instance_variable_set. Let's use them:

```
def add_checked_attribute(klass, attribute)
  klass.class_eval do
    define_method "#{attribute}=" do |value|
      raise 'Invalid attribute' unless value
      instance_variable_set("@#{attribute}", value)
    end

    define_method attribute do
      instance_variable_get "@#{attribute}"
    end
  end
end
```

"That's it," Bill says. "We now have a method that enters a class scope and defines instance methods that manipulate instance variables, and there's no string-based eval to speak of. Now that our code is both working and eval-free, we can move on to the third step in our development plan."

## Quiz: Checked Attributes (Step 3)

*Where you sprinkle some flexibility over today's project.*

To solve the boss' challenge, you and Bill still need to implement a few important features. One of these features is described in the third step of your development plan: "validate attributes through a block." Right now, your generated attribute raises an exception if you assign it nil or false. But it's supposed to support flexible validation through a block.

Because this step changes the interface of add_checked_attribute, it also calls for an update of the test suite. Bill replaces the two test cases that checked for nil or false attributes with a single new test case: